

Data Structure: 정렬 (1)

Prof. Jungmin Kwon
Spring, 2026

■ 학습목표

- 자료의 정렬 방법을 알아본다.
- 정렬 방법에 대한 알고리즘을 이해한다.
- 정렬 알고리즘의 효율을 알아본다.

■ 내용

01 정렬의 이해		02 선택 정렬
03 버블 정렬		04 퀵 정렬
05 삽입 정렬		06 셸 정렬
07 병합 정렬		08 기수 정렬
09 히프 정렬		10 트리 정렬

1. 정렬의 이해

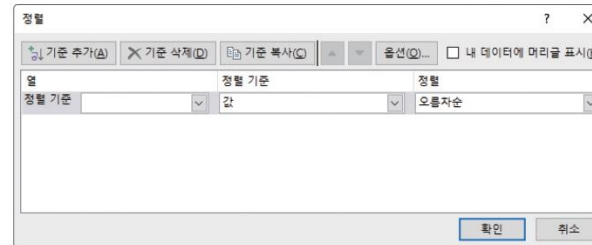
■ 정렬sort

- 순서 없이 배열된 자료를 작은 것부터 큰 것 순서인 오름차순Ascending이나 큰 것부터 작은 것 순서인 내림차순Descending으로 재배열하는 것
- 키(Key) - 자료를 정렬하는데 사용하는 기준이 되는 특정 값

- 정렬의 예



(a) 아이콘 정렬하기



(b) 데이터 시트에서 정렬하는 기준 정하기



(c) 쇼핑몰에서 제품 정렬하기

1. 정렬의 이해

■ 정렬 방식의 분류

표 9-1 정렬 방식의 분류

기준	정렬 방식	설명
실행 방법	비교식 정렬 ^{Comparative Sort}	비교할 각 키값을 한 번에 두 개씩 비교하여 교환함으로써 정렬을 실행하는 방식
	분배식 정렬 ^{Distribute Sort}	키값을 기준으로 하여 자료를 여러 개의 부분집합으로 분해하고, 각 부분집합을 정렬함으로써 전체를 정렬하는 방식
정렬 장소	내부 정렬 ^{Internal Sort}	컴퓨터 메모리 내부에서 정렬
	외부 정렬 ^{External Sort}	메모리의 외부인 보조 기억 장치에서 정렬

1. 정렬의 이해

– 내부 정렬^{internal sort}

- 정렬할 자료를 메인 메모리에 올려서 정렬하는 방식
- 정렬 속도가 빠르지만 정렬할 수 있는 자료의 양이 메인 메모리의 용량에 따라 제한됨

표 9-2 내부 정렬의 분류

구분	종류	설명
비교식	교환 방식	키를 비교하고 교환하여 정렬하는 방식(선택 정렬, 버블 정렬, 퀵 정렬)
	삽입 방식	키를 비교하고 삽입하여 정렬하는 방식(삽입 정렬, 셸 정렬)
	병합 방식	키를 비교하고 병합하여 정렬하는 방식(2-way 병합, n-way 병합)
	선택 방식	이진 트리를 사용하여 정렬하는 방식(히프 정렬, 트리 정렬)
분배식	분배 방식	키를 구성하는 값을 여러 개의 부분집합에 분배하여 정렬하는 방식(기수 정렬)

– 외부 정렬^{external sort}

데이터 일부만 주기억장치에 올리고, 나머지는 디스크 같은 보조기억장치에 둬

- 정렬할 자료를 보조 기억장치에서 정렬하는 방식
- 대용량의 보조 기억 장치를 사용하기 때문에 내부 정렬보다 속도는 떨어지지만 내부 정렬로 처리할 수 없는 대용량의 자료에 대한 정렬 가능
- 외부 정렬 방식
 - 병합 방식 : 파일을 부분 파일로 분리하여 각각을 내부 정렬 방법으로 정렬하여 병합하는 정렬 방식 (2-way 병합, n-way 병합)

2. 선택 정렬

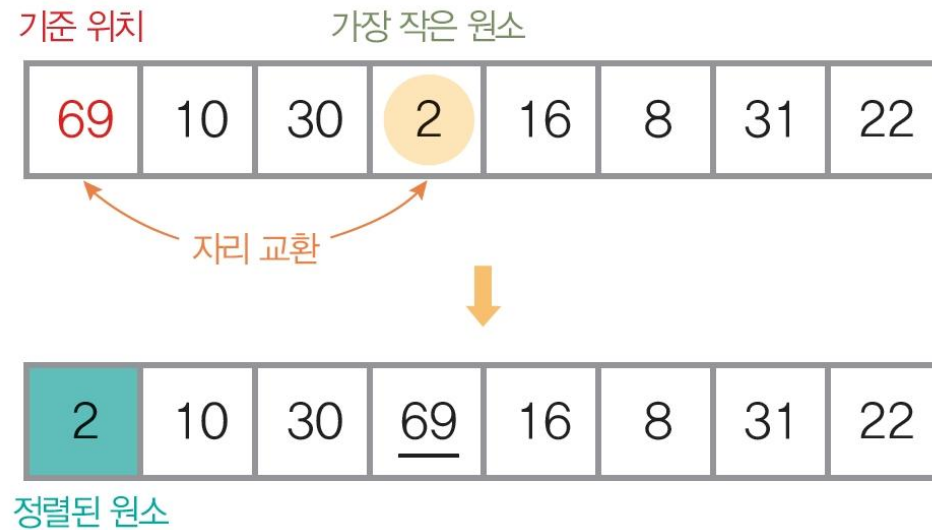
■ 선택 정렬 selection sort

- 전체 원소들 중에서 기준 위치에 맞는 원소를 선택하여 자리를 교환하는 방식으로 정렬
 - 해당 순서에 원소를 넣을 위치는 이미 정해져 있고, 어떤 원소를 넣을지 선택하는 알고리즘
 - Ex) 첫 번째 순서에는 최솟값을 넣는다
- 수행 방법
 - 전체 원소 중에서 가장 작은 원소를 찾아 첫 번째 원소와 자리를 교환
 - 두 번째로 작은 원소를 찾아 선택하여 두 번째 원소와 자리를 교환
 - 세 번째로 작은 원소를 찾아 선택하여 세 번째 원소와 자리를 교환
 - 이 과정을 반복하면서 정렬을 완성

2. 선택 정렬

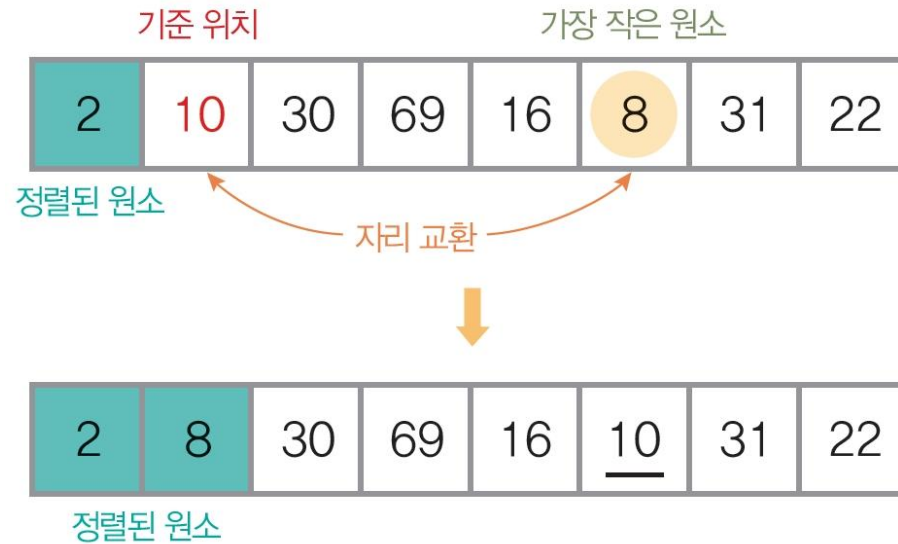
- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료를 선택 정렬 방법으로 정렬하는 과정

- ① 1단계 : 첫째 자리를 기준 위치로 정하고, 전체 중 가장 작은 원소 2를 선택한 후 기준 위치에 있는 원소 69와 자리를 교환



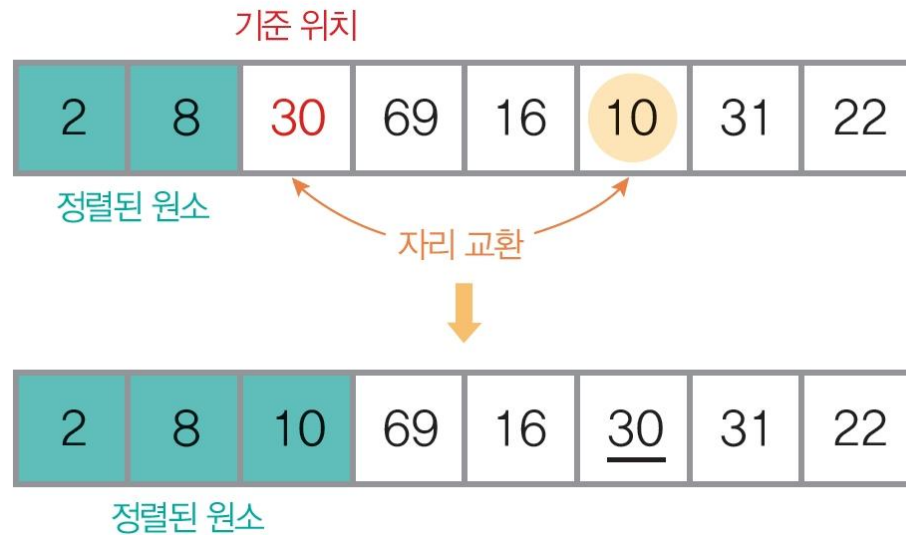
2. 선택 정렬

- ② 2단계 : 둘째 자리를 기준 위치로 정하고, 나머지 원소 중에서 가장 작은 원소인 8을 선택한 후 기준 위치에 있는 원소 10과 자리를 교환



2. 선택 정렬

- ③ 3단계 : 셋째 자리를 기준 위치로 정하고, 나머지 원소 중 가장 작은 원소인 10을 선택한 후 기준 위치에 있는 원소 30과 자리를 교환



2. 선택 정렬

- ④ 4단계 : 넷째 자리를 기준 위치로 정하고, 나머지 원소 중 가장 작은 원소인 16을 선택한 후 기준 위치에 있는 원소 69와 자리를 교환



2. 선택 정렬

- ⑤ 5단계 : 다섯째 자리를 기준 위치로 정하고, 나머지 원소 중에서 가장 작은 원소 22를 선택한 후 기준위치에 있는 원소 69와 자리를 교환



2. 선택 정렬

- ⑥ 6단계 : 여섯째 자리를 기준 위치로 정하고, 나머지 원소 중에서 가장 작은 원소 30을 선택한 후 기준위치에 있는 원소 30과 자리를 교환(제자리)



2. 선택 정렬

- ⑦ 7단계 : 일곱째 자리를 기준 위치로 정하고, 나머지 원소 중에서 가장 작은 원소 31을 선택한 후 기준 위치에 있는 원소 31과 자리를 교환(제자리)



- ⑧ 마지막에 남은 원소는 전체 원소 중에서 가장 큰 원소로, 마지막 자리에 남아 이미 정렬된 상태가 되므로 실행을 종료



2. 선택 정렬

- 선택 정렬 알고리즘
 - 크기가 n 인 배열 $a[]$ 의 원소를 선택 정렬하는 알고리즘

알고리즘 9-1 선택 정렬

```

selectionSort(a[], n)
  for ( $i \leftarrow 1$ ;  $i < n$ ;  $i \leftarrow i + 1$ ) do {
     $a[i], \dots, a[n - 1]$  중에서 가장 작은 원소  $a[k]$ 를 선택해  $a[i]$ 와 교환한다.
  }
end selectionSort()

```

2. 선택 정렬

- 메모리 사용공간

- n개의 원소에 대하여 n개의 메모리 사용

- 비교횟수

- 1단계 : 첫 번째 원소를 기준으로 n개의 원소 비교
- 2단계 : 두 번째 원소를 기준으로 마지막 원소까지 n-1개의 원소
- 3단계 : 세 번째 원소를 기준으로 마지막 원소까지 n-2개의 원소 비교
- i 단계 : i 번째 원소를 기준으로 n-i개의 원소 비교

n - 1 번비교

n - 2 번비교

- 전체 비교횟수 : $(n-1) + (n-2) + \dots + 2 + 1 = \sum_{i=1}^{n-1} n - i = \frac{n(n-1)}{2}$

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

- 어떤 경우에서나 비교횟수가 같으므로 시간 복잡도는 $O(n^2)$ 이 됨

예제 9-1

선택 정렬하기

[selectSort.c]

```
01  #include <stdio.h>
02  void SelectionSort(int a[], int size) {
03      int i, j, t, min, temp;
04
05      for (i = 0; i < size - 1; i++) {
06          min = i;  min : 가장 작은 원소 번호
07          for (j = i + 1; j < size; j++) {
08              if (a[j] < a[min]) min = j;
09          }
10          temp = a[i];      a[i] 와 a[min] 교환
11          a[i] = a[min];
12          a[min] = temp;
13          printf("\n%d단계 : ", i + 1);
14          for (t = 0; t < size; t++) printf("%3d ", a[t]);
15      }
16  }
```

[알고리즘 9-1] 구현

[ex9_1.c]

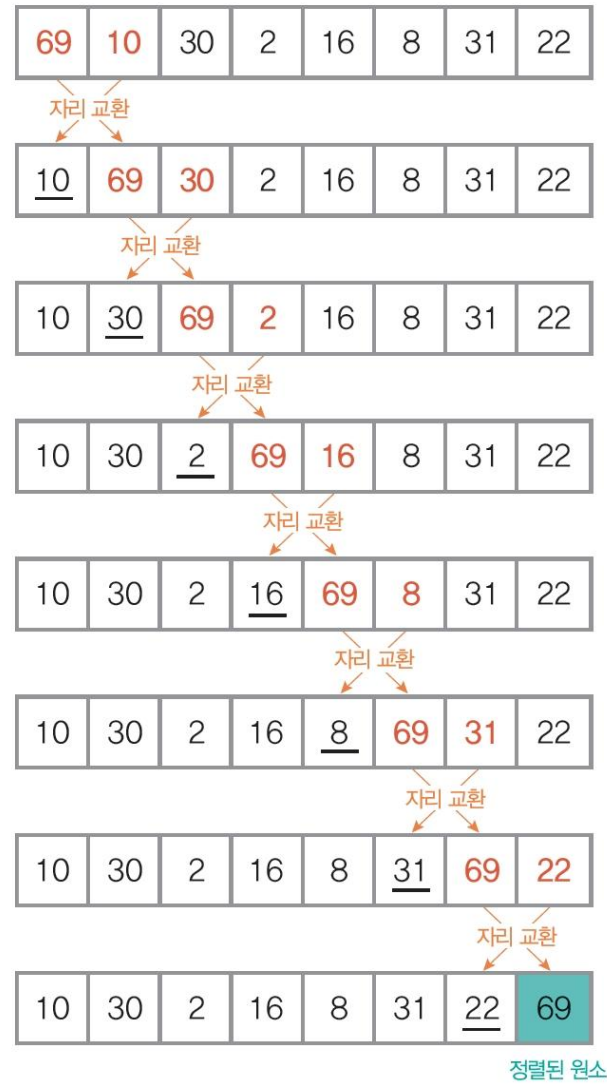
```
01  #include <stdio.h>
02  void SelectionSort(int a[], int size);
03
04  int main(void) {
05      int i, list[8] = { 69, 10, 30, 2, 16, 8, 31, 22 }; // 정렬할 원소
06      int size = sizeof(list)/ sizeof(list[0]);
07      printf("\n정렬할 원소 : ");
08      for (i = 0; i < size; i++) printf("%3d ", list[i]); // 정렬 전의 원소 출력
09      printf("\n<<<<<<<<<< 선택 정렬 수행 >>>>>>>>>>\n");
10      SelectionSort(list, size); // 선택 정렬 함수 호출
11
12      getchar(); return 0;
13  }
```

3. 버블 정렬

■ 버블 정렬 bubble sort의 이해

- 인접한 두 개의 원소를 비교하여 자리를 교환하는 방식
 - 첫 번째 원소부터 마지막 원소까지 반복하여 한 단계가 끝나면 가장 큰 원소가 마지막 자리로 정렬
 - 첫 번째 원소부터 인접한 원소끼리 계속 자리를 교환하면서 맨 마지막 자리로 이동하는 모습이 물 속에서 물 위로 올라오는 물방울 모양과 같다고 하여 버블 bubble 정렬이라 함.
- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료들을 버블 정렬 방법으로 정렬하는 과정
 - ① 1단계 : 인접한 두 원소를 비교하여 자리를 교환하는 작업을 첫 번째 원소부터 마지막 원소까지 차례로 반복하여 가장 큰 원소 69를 마지막 자리로 정렬.

3. 버블 정렬



3. 버블 정렬

② 2단계 : 정렬하지 않은 원소에 대해 두 번째 버블 정렬을 수행하면, 나머지 원소 중에서 가장 큰 원소 31이 끝에서 둘째 자리로 정렬

10	30	2	16	8	31	22	69
----	----	---	----	---	----	----	----

10	30	2	16	8	31	22	69
----	----	---	----	---	----	----	----

자리 교환

10	<u>2</u>	30	16	8	31	22	69
----	----------	----	----	---	----	----	----

자리 교환

10	2	<u>16</u>	30	8	31	22	69
----	---	-----------	----	---	----	----	----

자리 교환

10	2	16	<u>8</u>	30	31	22	69
----	---	----	----------	----	----	----	----

10	2	16	8	30	31	22	69
----	---	----	---	----	----	----	----

자리 교환

10	2	16	8	30	<u>22</u>	31	69
----	---	----	---	----	-----------	----	----

정렬된 원소

3. 버블 정렬

- ③ 3단계 : 세 번째 버블 정렬을 수행하면, 나머지 원소 중에서 가장 큰 원소 30이 끝에서 셋째 자리로 정렬

10	2	16	8	30	22	31	69
----	---	----	---	----	----	----	----

자리 교환

<u>2</u>	10	16	8	30	22	31	69
----------	----	----	---	----	----	----	----

2	10	16	8	30	22	31	69
---	----	----	---	----	----	----	----

자리 교환

2	10	<u>8</u>	16	30	22	31	69
---	----	----------	----	----	----	----	----

2	10	8	16	30	22	31	69
---	----	---	----	----	----	----	----

자리 교환

2	10	8	16	<u>22</u>	30	31	69
---	----	---	----	-----------	----	----	----

정렬된 원소

3. 버블 정렬

- ④ 4단계 : 네 번째 버블 정렬을 정렬하지 않은 원소에 대해 수행하면, 나머지 원소 중에서 가장 큰 원소 22가 끝에서 넷째 자리로 정렬

2	10	8	16	22	30	31	69
---	----	---	----	----	----	----	----

2	10	8	16	22	30	31	69
---	----	---	----	----	----	----	----

자리 교환

2	<u>8</u>	10	16	22	30	31	69
---	----------	----	----	----	----	----	----

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

정렬된 원소

3. 버블 정렬

- ⑤ 5단계 : 다섯 번째 버블 정렬을 수행하면, 나머지 원소 중에서 가장 큰 원소 16이 끝에서 다섯째 자리로 정렬

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

정렬된 원소

3. 버블 정렬

- ⑥ 6단계 : 여섯 번째 버블 정렬을 수행하면, 나머지 원소 중에서 가장 큰 원소 10이 끝에서 여섯째 자리로 정렬

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

정렬된 원소

3. 버블 정렬

- ⑦ 7단계 : 나머지 원소 중에서 가장 큰 원소 8이 끝에서 일곱째 자리로 정렬

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

정렬된 원소

- ⑧ 마지막에 남은 첫째 원소는 전체 원소 중에서 가장 작은 원소로, 가장 앞자리에 남아 이미 정렬된 상태가 되므로 실행을 종료

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----



2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

정렬된 원소

3. 버블 정렬

■ 버블 정렬 알고리즘

- 크기가 n 인 배열 $a[]$ 의 원소를 버블 정렬하는 알고리즘

처음 정렬할 원소 번호

$i = n-1$

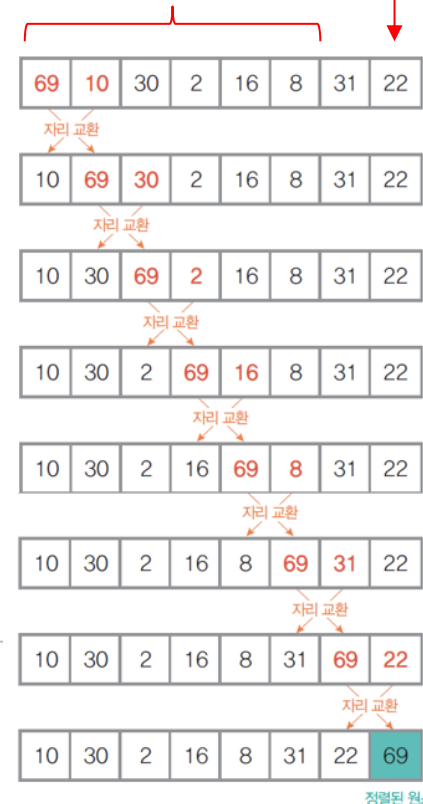
알고리즘 9-2 버블 정렬

j 범위

```

bubbleSort(a[], n)
  for (i ← n - 1; i ≥ 0; i ← i - 1) do {
    for (j ← 0; j < i; j ← j + 1) do {
      if (a[j] > a[j + 1]) then {
        temp ← a[j];
        a[j] ← a[j + 1];
        a[j + 1] ← temp;
      }
    }
  }
end bubbleSort()
  
```

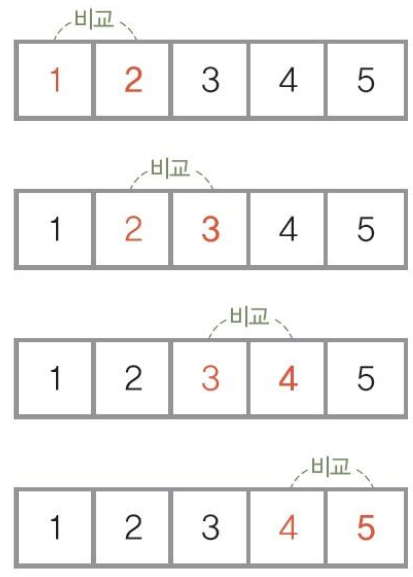
첫 단계



3. 버블 정렬

- 메모리 사용공간
 - n개의 원소에 대하여 n개의 메모리 사용
- 연산 시간
 - 최선의 경우 : 자료가 이미 정렬되어있는 경우
 - 비교횟수 : i번째 원소를 (n-i)번 비교하므로, $\frac{n(n-1)}{2}$ 번
 - 자리교환횟수 : 자리교환이 발생하지 않음
 - 최악의 경우 : 자료가 역순으로 정렬되어있는 경우
 - 비교횟수 : i번째 원소를 (n-i)번 비교하므로, $\frac{n(n-1)}{2}$ 번
 - 자리교환횟수 : i번째 원소를 (n-i)번 교환하므로, $\frac{n(n-1)}{2}$ 번
 - 최선의 경우와 최악의 경우에 대한 평균 시간 복잡도를 빅-오 Big Oh 표기법으로 나타내면 $O(n^2)$ 이 됨

3. 버블 정렬



- 전체 비교 횟수 : $1+2+3+\dots+(n-1) = \frac{n(n-1)}{2}$
- 전체 자리 교환 횟수 : 0
- 시간 복잡도 : $O(n^2)$

(a) 순차 정렬된 경우 : 최선의 경우



- 전체 비교 횟수 : $1+2+3+\dots+(n-1) = \frac{n(n-1)}{2}$
- 전체 자리 교환 횟수 : $1+2+3+\dots+(n-1) = \frac{n(n-1)}{2}$
- 시간 복잡도 : $O(n^2)$

(b) 역순 정렬된 경우 : 최악의 경우

그림 9-2 순차 정렬된 경우와 역순 정렬된 경우의 버블 정렬 비교

3. 버블 정렬

- [예제 9-2] 버블 정렬하기 : [교재 525p](#)
- 실행 결과

```

정렬할 원소: 69 10 30 2 16 8 31 22

<<<<<<<<< 버블 정렬 수행 >>>>>>>>>>>>

1단계>>
  10 69 30 2 16 8 31 22
  10 30 69 2 16 8 31 22
  10 30 2 69 16 8 31 22
  10 30 2 16 69 8 31 22
  10 30 2 16 8 69 31 22
  10 30 2 16 8 31 69 22
  10 30 2 16 8 31 22 69

2단계>>
  10 30 2 16 8 31 22 69
  10 2 30 16 8 31 22 69
  10 2 16 30 8 31 22 69
  10 2 16 8 30 31 22 69
  10 2 16 8 30 31 22 69
  10 2 16 8 30 22 31 69

```

```

3단계>>
  2 10 16 8 30 22 31 69
  2 10 16 8 30 22 31 69
  2 10 8 16 30 22 31 69
  2 10 8 16 30 22 31 69
  2 10 8 16 22 30 31 69

4단계>>
  2 10 8 16 22 30 31 69
  2 8 10 16 22 30 31 69
  2 8 10 16 22 30 31 69
  2 8 10 16 22 30 31 69

5단계>>
  2 8 10 16 22 30 31 69
  2 8 10 16 22 30 31 69
  2 8 10 16 22 30 31 69

6단계>>
  2 8 10 16 22 30 31 69
  2 8 10 16 22 30 31 69

7단계>>
  2 8 10 16 22 30 31 69

```

3. 버블 정렬

예제 9-2 버블 정렬하기

[bubbleSort.c]

```
01  #include <stdio.h>
02  void bubbleSort(int a[], int size) {
03      int i, j, t, temp;
04
05      for (i = size - 1; i > 0; i--) {
06          printf("\n %d단계>>", size - i);
07          for (j = 0; j < i; j++) {
08              if (a[j] > a[j + 1]) {
09                  temp = a[j];
10                  a[j] = a[j + 1];
11                  a[j + 1] = temp;
12              }
13              printf("\n\t");
14              for (t = 0; t < size; t++) printf("%3d ", a[t]);
15          }
16      }
17  }
```

[알고리즘 9-2] 구현

$i = n-1$
 $= \text{size}-1$

처음교환과정

$j = 0$ 교환결과

$j = 1$ 교환결과



3. 버블 정렬

[ex9_2.c]

```

01  #include <stdio.h>
02  void bubbleSort(int a[], int size);
03
04  int main(void) {
05      int i, list[8] = { 69, 10, 30, 2, 16, 8, 31, 22 };    // 정렬할 원소
06      int size = sizeof(list) / sizeof(list[0]);
07      printf("\n정렬할 원소 : ");
08      for (i = 0; i < size; i++) printf("%3d ", list[i]);  // 정렬 전의 원소 출력
09      printf("\n<<<<<<<<< 버블 정렬 수행 >>>>>>>>>");
10      bubbleSort(list, size);                                // 버블 정렬 함수 호출
11
12      getchar(); return 0;
13  }

```

정렬할 원소 : 69 10 30 2 16 8 31 22

<<<<<<<<< 버블 정렬 수행 >>>>>>>>>

1단계>>

```

10 69 30 2 16 8 31 22
10 30 69 2 16 8 31 22
10 30 2 69 16 8 31 22
10 30 2 16 69 8 31 22
10 30 2 16 8 69 31 22
10 30 2 16 8 31 69 22
10 30 2 16 8 31 22 69

```

2단계>>

```

10 30 2 16 8 31 22 69
10 2 30 16 8 31 22 69
10 2 16 30 8 31 22 69
10 2 16 8 30 31 22 69
10 2 16 8 30 31 22 69
10 2 16 8 30 22 31 69

```

3단계>>

```

2 10 16 8 30 22 31 69
2 10 16 8 30 22 31 69
2 10 8 16 30 22 31 69
2 10 8 16 30 22 31 69
2 10 8 16 22 30 31 69

```

4단계>>

```

2 10 8 16 22 30 31 69
2 8 10 16 22 30 31 69
2 8 10 16 22 30 31 69
2 8 10 16 22 30 31 69

```

5단계>>

```

2 8 10 16 22 30 31 69
2 8 10 16 22 30 31 69
2 8 10 16 22 30 31 69

```

6단계>>

```

2 8 10 16 22 30 31 69
2 8 10 16 22 30 31 69

```

7단계>>

```

2 8 10 16 22 30 31 69

```


1. 학생 자료를 저장하기 위한 구조체 struct student 를 선언한다.
이 구조체에는 이름과 국어성적 수학성적 멤버변수가 있다.
이 구조체를 새로운 자료형 Student 로 선언(typedef)한다.
2. 예제9-2 bubbleSort() 함수를 수정하여
bubbleSort_new(Student a[], int size, int key) 를 작성하시오.
이 함수는 학생들의 자료를 key 값의 순서로 작은 값 부터 큰 값 순으로 정렬한다.
key=1 이면 국어성적순으로 key = 2 이면 수학성적 순으로 정렬한다.
3. main() 함수에서 Student 자료형 배열 5개 크기를 선언하고 임의의 값으로 초기화 한다.
학생의 자료를 배열에 저장된 순서로 화면에 출력한다.
사용자에게 정렬 key 값을 물어보고 입력 받아서 bubbleSort_new() 함수를 호출하여 정렬 결과를 출력한다.

```
Student list[5] = {  
    {"Kim", 90, 85},  
    {"Lee", 75, 95},  
    {"Park", 88, 70},  
    {"Choi", 60, 80},  
    {"Jung", 95, 92}  
};
```

4. 퀵 정렬

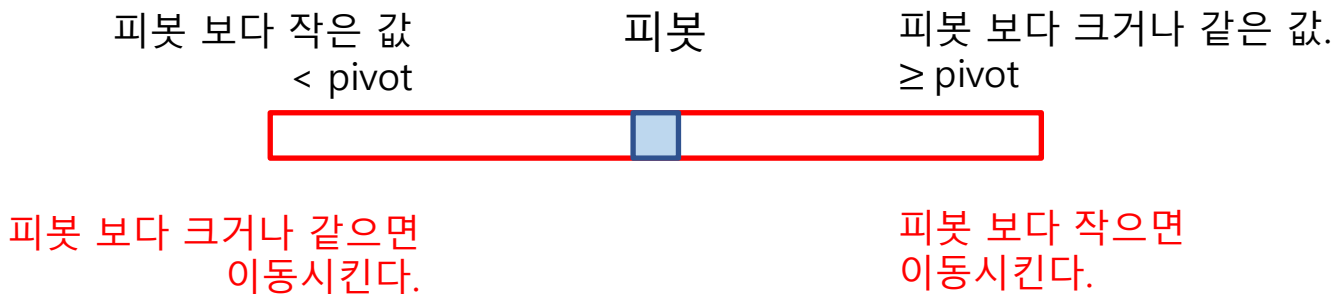
■ 퀵 정렬 quick sort의 이해

- 정렬할 전체 원소에 대해서 정렬을 수행하지 않고, 기준 값을 중심으로 왼쪽 부분 집합과 오른쪽 부분 집합으로 분할하여 정렬하는 방법
 - 왼쪽 부분 집합에는 기준 값보다 작은 원소들을 이동시키고, 오른쪽 부분 집합에는 기준 값보다 큰 원소들을 이동시킴
 - 기준 값 : 피벗 pivot
 - 일반적으로 전체 원소 중에서 가운데에 위치한 원소를 선택
- 퀵 정렬은 다음의 두 가지 기본 작업을 반복 수행하여 완성
 - 분할 divide
 - 정렬할 자료들을 기준값을 중심으로 두 개로 나눠 부분집합을 만듦
 - 정복 conquer
 - 부분집합 안에서 기준값의 정렬 위치를 정함

4. 퀵 정렬

■ 퀵 정렬 동작 규칙

- 중앙 위치의 원소를 피벗(pivot) 으로 사용하기로 한다.
- 피벗의 왼쪽에는 피벗 보다 작은 값의 원소들이 있어야 한다.
- 피벗의 오른쪽에는 피벗 보다 크거나 같은 값의 원소들이 있어야 한다.
- 위 기준에 맞지 않는 원소(이동 할 원소)는 피벗 반대 편으로 이동시킨다.
 - 피벗 양쪽에 이동 할 원소가 있으면 서로 위치를 교환한다.
 - 피벗 한쪽에만 이동 할 원소가 있으면 피벗과 이동할 원소의 위치를 교환한다.
이 경우는 피벗 한쪽에는 정렬이 완료된 경우 이므로 교환 이후 피벗의 원소는 자리가 확정된다.



4. 퀵 정렬

- 퀵 정렬 동작 규칙

L : 왼쪽에서 제거할 원소
R : 오른쪽에서 제거할 원소

- ① 왼쪽 끝에서 오른쪽으로 움직이면서 크기를 비교하여 피벗보다 크거나 같은 원소를 찾아 L로 표시한다. 단, L은 R과 만나면 더 이상 오른쪽으로 이동하지 못하고 멈춘다.
- ② 오른쪽 끝에서 왼쪽으로 움직이면서 피벗보다 작은 원소를 찾아 R로 표시한다. 단, R은 L과 만나면 더 이상 왼쪽으로 이동하지 못하고 멈춘다.
- ③-a ①과 ②에서 찾은 L 원소와 R 원소가 있는 경우, 서로 교환하고 L과 R의 현재 위치에서 ①과 ② 작업을 다시 수행한다.
- ③-b ① ~ ②를 수행하면서 L과 R이 같은 원소에서 만나 멈춘 경우, 피벗과 R의 원소를 서로 교환한다. 교환된 자리를 피벗 위치로 확정하고 현재 단계의 퀵 정렬을 끝낸다.
- ④ 피벗의 확정된 위치를 기준으로 만들어진 새로운 왼쪽 부분집합과 오른쪽 부분집합에 대해서 ① ~ ③의 퀵 정렬을 순환적으로 반복 수행하는데, 모든 부분집합의 크기가 1 이하가 되면 전체 퀵 정렬을 종료한다.

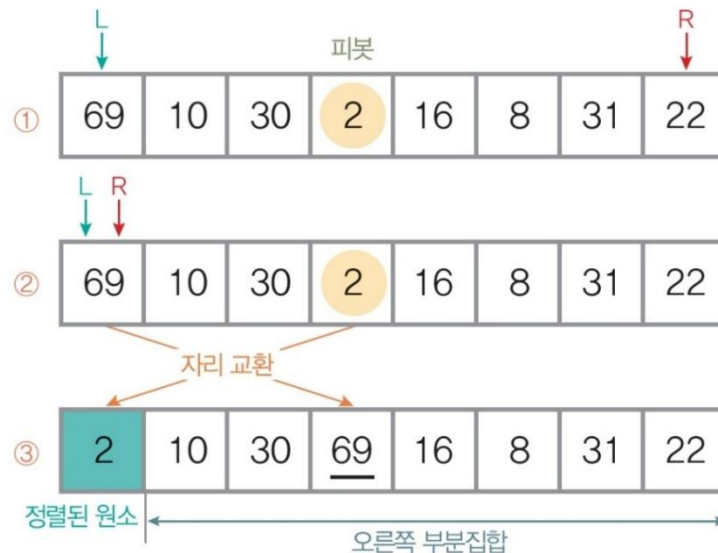
그림 9-3 퀵 정렬을 위한 동작 규칙

4. 퀵 정렬

- 정렬하지 않은 {69, 10, 30, 2, 16, 8, 31, 22} 자료를 퀵 정렬 방법으로 정렬하는 과정 (가운데 원소를 피봇으로 정함)

(1) 1단계 :

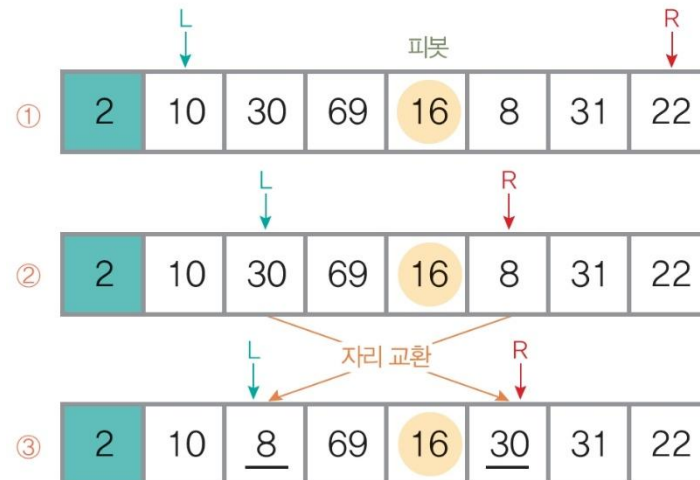
- ① 원소 2를 피봇으로 선택하고 퀵 정렬을 시작
- ② L은 정렬 범위의 왼쪽 끝에서 오른쪽으로 움직이면서 피봇보다 크거나 같은 원소를 찾고, R은 정렬 범위의 오른쪽 끝에서 왼쪽으로 움직이면서 피봇보다 작은 원소를 찾음. L은 원소 69를 찾았지만, R은 피봇보다 작은 원소를 찾지 못한 상태로 원소 69에서 L과 만남. L과 R이 만나 더 이상 진행할 수 없는 상태가 되었으므로,
- ③ 원소 69를 피봇과 자리를 교환하고 피봇 원소 2의 위치를 확정



4. 퀵 정렬

(2) 2단계 : 위치가 확정된 피벗 2의 왼쪽 부분집합은 공집합이므로 퀵 정렬을 수행하지 않고, 오른쪽 부분집합에 대해서 퀵 정렬을 수행.

- ① 오른쪽 부분집합의 원소가 일곱 개이므로 가운데 있는 원소 16을 피벗으로 선택하고 퀵 정렬을 시작.
- ② L은 오른쪽으로 움직이면서 피벗보다 크거나 같은 원소인 30을 찾고, R은 왼쪽으로 움직이면서 피벗보다 작은 원소인 8을 찾음.
- ③ L이 찾은 30과 R이 찾은 8을 서로 자리를 교환한 후 현재 위치에서 L은 다시 오른쪽으로 움직이면서 피벗보다 크거나 같은 원소 69를 찾고 R은 피벗보다 작은 원소를 찾음.



4. 퀵 정렬

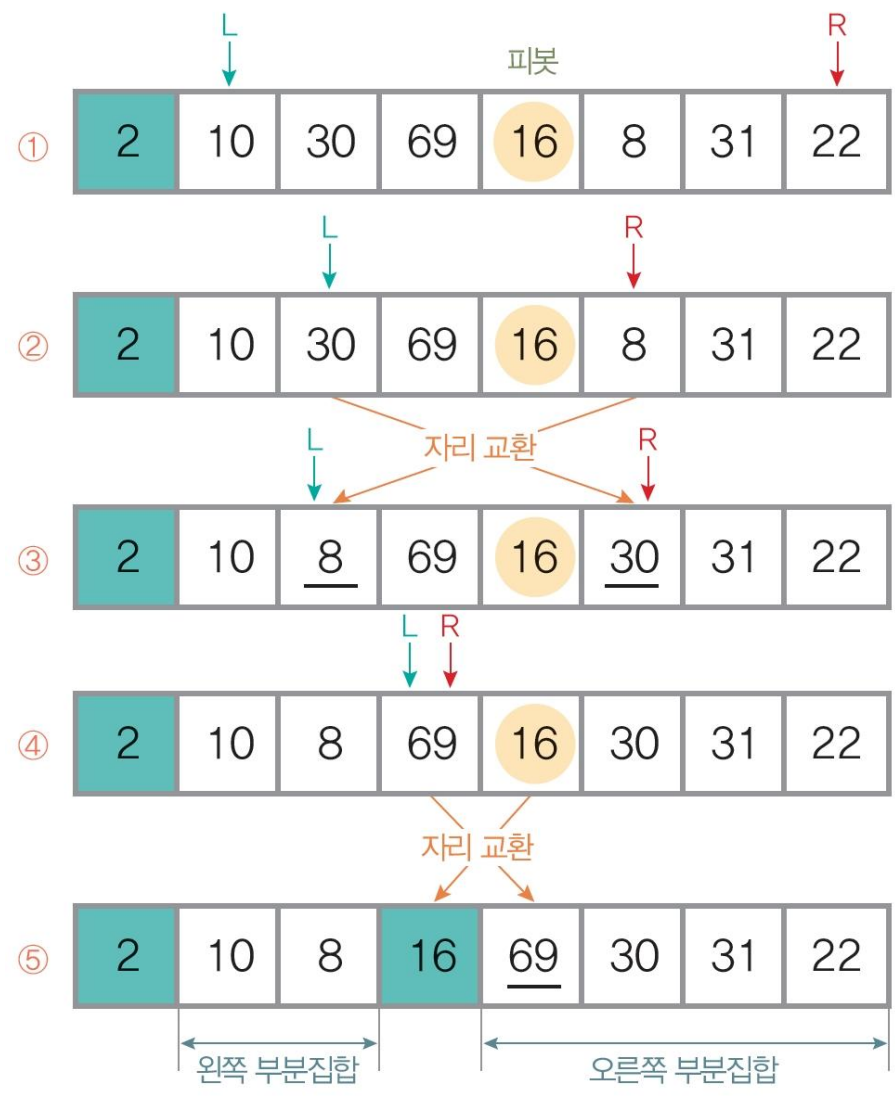
(2) 2단계 : 위치가 확정된 피벗 2의 왼쪽 부분집합은 공집합이므로 퀵 정렬을 수행하지 않고, 오른쪽 부분집합에 대해서 퀵 정렬을 수행.

④ R이 원소 69에서 L과 만나 더 이상 진행할 수 없는 상태가 되었으므로,

⑤ 원소 69를 피벗과 교환하고 피벗 원소 16의 위치를 확정



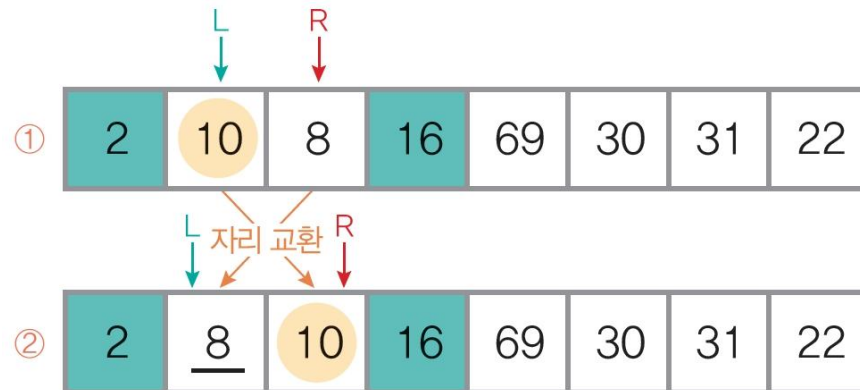
4. 퀵 정렬



4. 퀵 정렬

(3) 3단계 : 위치가 확정된 피벗 16을 기준으로 새로 생긴 왼쪽 부분집합에서 퀵 정렬을 수행

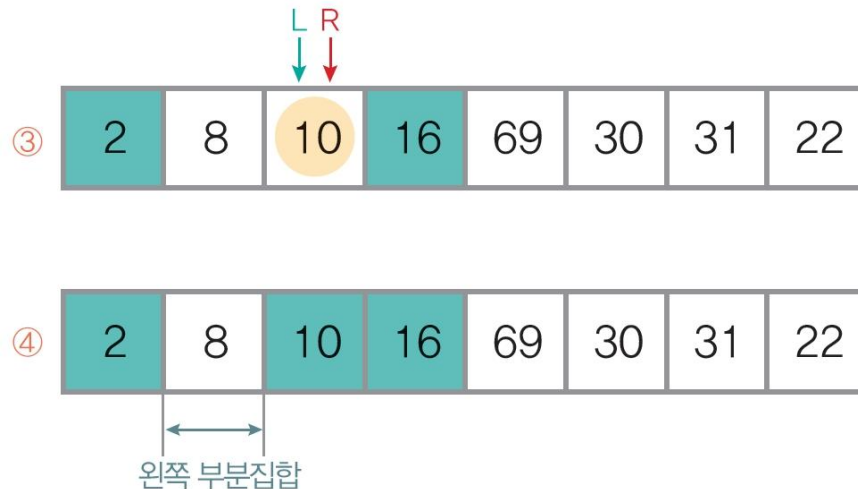
- ① 원소 10을 피벗으로 선택하고 L은 오른쪽으로 움직이면서 피벗보다 크거나 같은 원소를, R은 왼쪽으로 움직이면서 피벗보다 작은 원소를 찾음
- ② L이 찾은 10과 R이 찾은 8을 서로 교환



4. 퀵 정렬

(3) 3단계 : 위치가 확정된 피벗 16을 기준으로 새로 생긴 왼쪽 부분집합에서 퀵 정렬을 수행

- ③ 현재 위치에서 L은 다시 오른쪽으로 움직이다가 원소 10에서 R과 만나서 멈추게 됨. 더 이상 진행할 수 없는 상태가 되었으므로,
- ④ R의 원소와 피벗을 교환하고 피벗 원소 10의 위치를 확정한다(이 경우에는 R과 피벗 위치가 같았으므로, 자리를 교환하기 전과 후의 상태가 같음)



4. 퀵 정렬

(4) 4단계 : 피벗 10의 왼쪽 부분집합은 원소가 한 개이므로 퀵 정렬을 수행하지 않고, 오른쪽 부분집합은 공집합이므로 역시 퀵 정렬을 수행하지 않음
이제 [2]에서 피벗이었던 원소 16의 오른쪽 부분집합에 대해서 퀵 정렬을 수행

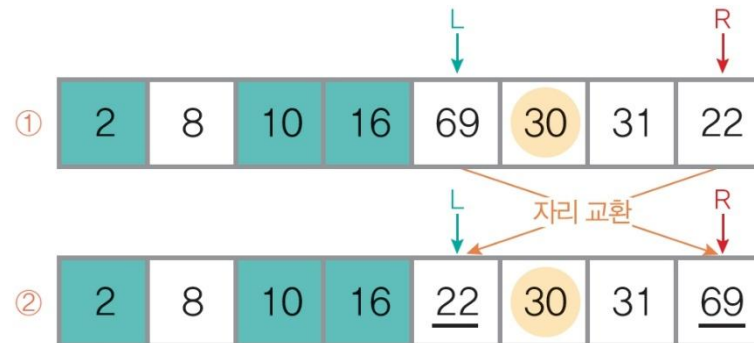
- ① 오른쪽 부분집합의 원소가 네 개이므로 가운데 있는 원소 30을 피벗으로 선택. L은 오른쪽으로 이동하면서 피벗보다 크거나 같은 원소를 찾고, R은 왼쪽으로 움직이면서 피벗보다 작은 원소를 찾음



4. 퀵 정렬

(4) 4단계 : 피봇 10의 왼쪽 부분집합은 원소가 한 개이므로 퀵 정렬을 수행하지 않고, 오른쪽 부분집합은 공집합이므로 역시 퀵 정렬을 수행하지 않음
이제 [2]에서 피봇이었던 원소 16의 오른쪽 부분집합에 대해서 퀵 정렬을 수행

② L이 찾은 69와 R이 찾은 22를 서로 교환. 현재 위치에서 다시 L은 피봇보다 크거나 같은 원소를 찾아 오른쪽으로 움직이고 R은 피봇보다 작은 원소를 찾아 왼쪽으로 움직이다가



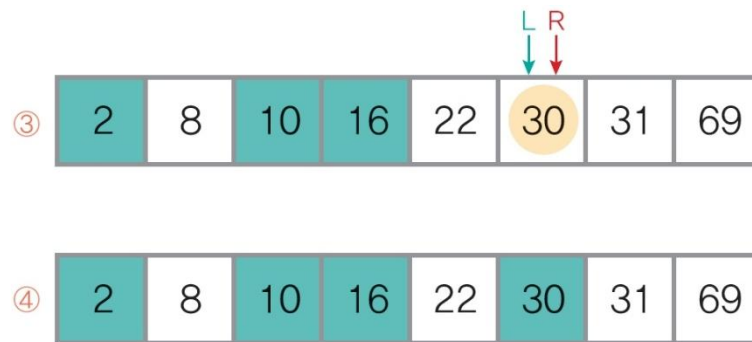
4. 퀵 정렬

(4) 4단계 : 피벗 10의 왼쪽 부분집합은 원소가 한 개이므로 퀵 정렬을 수행하지 않고, 오른쪽 부분집합은 공집합이므로 역시 퀵 정렬을 수행하지 않음
이제 [2]에서 피벗이었던 원소 16의 오른쪽 부분집합에 대해서 퀵 정렬을 수행

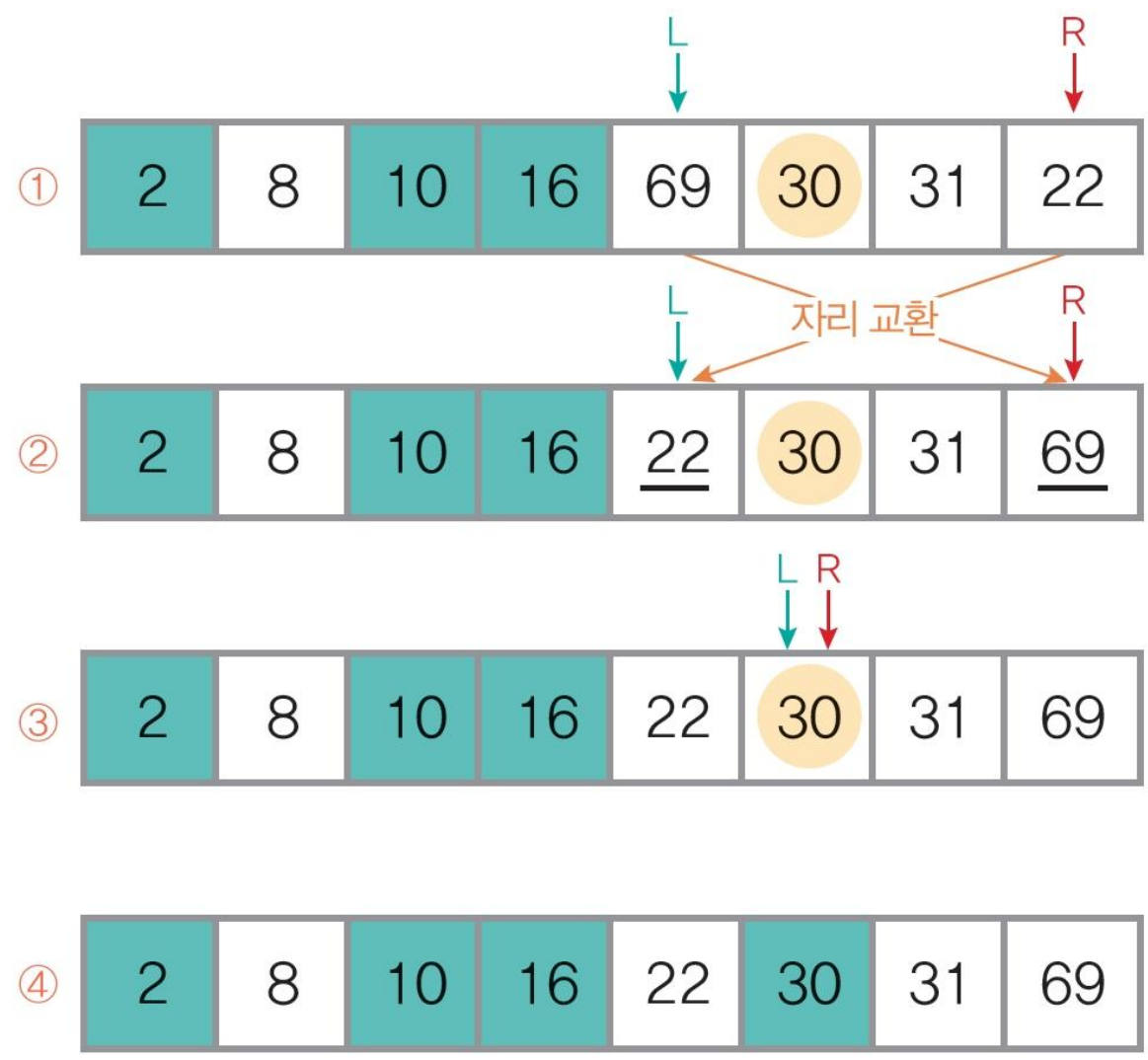
③ 원소 30에서 L과 R이 만나 멈춤. 더 이상 진행할 수 없으므로

④ R의 원소와 피벗을 교환하고 피벗 원소 30의 위치가 확정

(이 경우에 R과 피벗 위치가 같았으므로, 자리를 교환하기 전과 후의 상태가 같음).



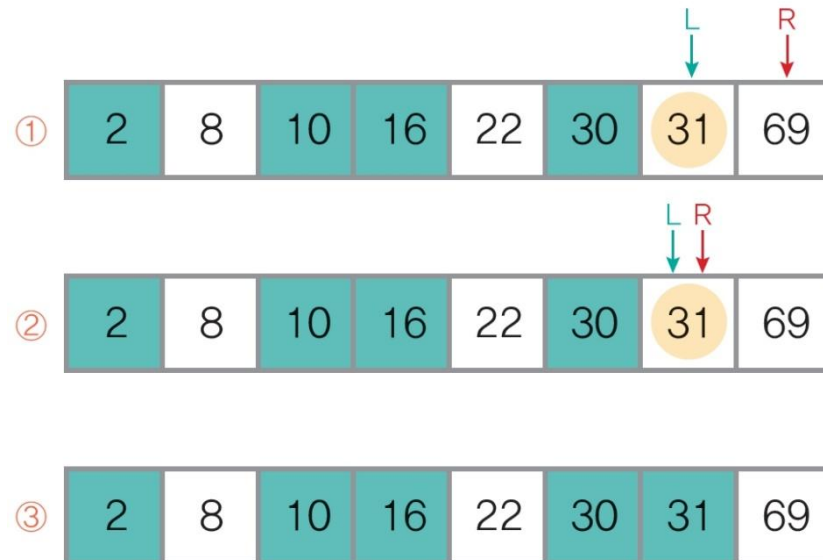
4. 퀵 정렬



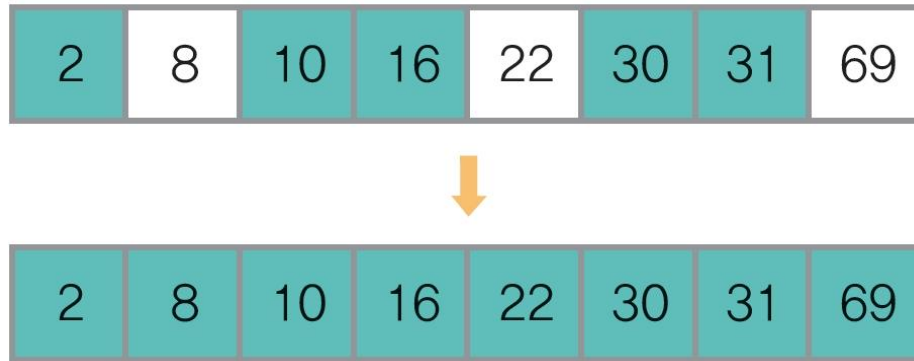
4. 퀵 정렬

(5) 5단계 : 피벗 30의 왼쪽 부분집합의 원소가 한 개이므로 퀵 정렬을 수행하지 않고, 오른쪽 부분집합에 대해서 퀵 정렬을 수행

- ① 이때 피벗은 31을 선택하고 L은 오른쪽으로 움직이면서 피벗보다 크거나 같은 원소를 찾고 R은 왼쪽으로 움직이면서 피벗보다 작은 원소를 찾음
- ② L과 R이 원소 31에서 만나 더 이상 진행하지 못하는 상태가 되어
- ③ 원소 31을 피벗과 교환하여 위치를 확정(이 경우에는 R과 피벗 위치가 같았으므로, 자리를 교환하기 전과 후의 상태가 같음).



- (6) 6단계 : 피벗 31의 오른쪽 부분집합의 원소가 한 개이므로 퀵 정렬을 수행하지 않음. 모든 부분 집합의 크기가 1 이하이므로 전체 퀵 정렬을 종료



4. 퀵 정렬

- 퀵 정렬 알고리즘
 - 크기가 n 인 배열 $a[]$ 의 원소를 퀵 정렬하는 알고리즘

알고리즘 9-3 퀵 정렬

```

quickSort(a[], begin, end)
    if (begin < end) {
        p ← partition(a, begin, end); // 피벗 위치를確定하여 분할 위치를 정한다.
        quickSort(a[], begin, p-1); // 피벗 왼쪽부분에 대해 퀵정렬
        quickSort(a[], p+1, end); // 피벗 오른쪽 부분에 대해 퀵정렬
    }
end quickSort()

```

교재 오류 수정할 것 !

4. 퀵 정렬

- 퀵 정렬 알고리즘은 배열 $a[]$ 를 두 개로 분할하는 `partition()` 연산이 필요

알고리즘 9-4 파티션 분할

```

partiton(a[], begin, end)
    pivot ← (begin + end) / 2;
    L ← begin;
    R ← end;
    while (L < R) do {
        while (a[L] < a[pivot] and L < R) do L++;
        while (a[R] ≥ a[pivot] and L < R) do R--;
        if (L < R) then { // L의 원소와 R의 원소 교환
            temp ← a[L];
            a[L] ← a[R];
            a[R] ← temp;
        }
    }
    temp ← a[pivot]; // R의 원소와 피벗 원소 교환
    a[pivot] ← a[R];
    a[R] ← temp;
    return R; // 교환된 R의 자리를 분할 기준 자리로 반환
end partiton()

```

4. 퀵 정렬

- 메모리 사용공간
 - n 개의 원소에 대하여 n 개의 메모리 사용
- 연산 시간
 - 최선의 경우
 - 피벗에 의해서 원소들이 왼쪽 부분 집합과 오른쪽 부분 집합으로 정확히 $n/2$ 개씩 2등분이 되는 경우가 반복되어 수행 단계 수가 최소가 되는 경우
 - 최악의 경우
 - 피벗에 의해 원소들을 분할하였을 때 한 개와 $n-1$ 개로 한쪽으로 치우쳐 분할되는 경우가 반복되어 수행 단계 수가 최대가 되는 경우
 - 평균 시간 복잡도 : $O(n \log_2 n)$
 - 같은 시간 복잡도를 가지는 다른 정렬 방법에 비해서 자리 교환 횟수를 줄임으로써 더 빨리 실행되어 실행 시간 성능이 좋은 정렬 방법임.

예제 9-3 퀵 정렬하기 1

[quickSort1.c]

```

01  #include <stdio.h>
02
03  int i = 0;           // 퀵 정렬 단계 출력용 변수
04
05  // 주어진 부분집합 안에서 피벗의 위치를確定하여 분할 위치를 정하는 연산 [알고리즘 9-4] 수정 구현
06  int partition(int a[], int begin, int end, int size) {
07      int pivot, L, R, t, temp;
08      L = begin;
09      R = end;
10      pivot = (begin + end) / 2; // 중간에 위치한 원소를 피벗 원소로 선택
11      printf("\n [%d단계 : pivot = %d ] \n", ++i, a[pivot]);
12      while (L < R) {
13          while ((a[L] < a[pivot]) && (L < R)) L++;
14          while ((a[R] >= a[pivot]) && (L < R)) R--;
15          if (L < R) { // L과 R 원소의 자리 교환
16              temp = a[L];
17              a[L] = a[R];
18              a[R] = temp;
19              // L이 피벗인 경우, 변경된 피벗의 위치(R)를 다시 저장!
20              if (L == pivot) pivot = R;
21          } // if(L<R)
22      } // while(L<R)

```

int step=0; 으로 변경

++step 로 변경

```
23 // (L=R)이 된 경우,  
24 // 더 이상 진행할 수 없으므로 R 원소와 피벗 원소의 자리를 교환하여 마무리  
25 temp = a[pivot];  
26 a[pivot] = a[R];  
27 a[R] = temp;  
28 for (t = 0; t < size; t++) printf("%4d", a[t]); // 현재의 정렬 상태 출력  
29 printf("\n");  
30 return R; // 정렬되어 확정된 피벗의 위치 반환  
31 }  
32  
33 void quickSort(int a[], int begin, int end, int size) { [알고리즘 9-3] 구현  
34     int p;  
35     if (begin < end) {  
36         p = partition(a, begin, end, size); // 피벗의 위치에 의해 분할 위치 결정  
37         quickSort(a, begin, p - 1, size); // 피벗의 왼쪽 부분집합에 대해 퀵 정렬을 재귀호출  
38         quickSort(a, p + 1, end, size); // 피벗의 오른쪽 부분집합에 대해 퀵 정렬을 재귀호출  
39     }  
40 }
```

4. 퀵 정렬

[ex9_3.c]

```

01  #include <stdio.h>
02  void quickSort(int a[], int begin, int end, int size);
03
04  int main(void) {
05      int i, list[8] = { 69, 10, 30, 2, 16, 8, 31, 22 };
06      int size = sizeof(list) / sizeof(list[0]);    // list 배열의 원소 개수
07      printf("\n 정렬할 원소 : ");
08      for (i = 0; i < size; i++) printf("%3d", list[i]);
09      printf("\n\n<<<<<<<<< 퀵 정렬 수행 >>>>>>>>>\n");
10      quickSort(list, 0, size - 1, size);           // 퀵 정렬 함수 호출
11
12      getchar(); return 0;
13  }

```

정렬할 원소 : 69 10 30 2 16 8 31 22

<<<<<<<<< 퀵 정렬 수행 >>>>>>>>>

[1단계 : pivot=2]

2 10 30 69 16 8 31 22

[2단계 : pivot=16]

2 10 8 16 69 30 31 22

[3단계 : pivot=10]

2 8 10 16 69 30 31 22

[4단계 : pivot=30]

2 8 10 16 22 30 31 69

[5단계 : pivot=31]

2 8 10 16 22 30 31 69

4. 퀵 정렬

- [예제 9-4] 퀵 정렬하기 2 : [교재 535p](#)
 - 피벗 위치를 첫째 원소로 정하여 퀵 정렬
- 실행 결과

정렬할 원소 : 69 10 30 2 16 8 31 22

<<<<<<<< 퀵 정렬 수행 >>>>>>>>>

[1단계 : pivot=69]

22 10 30 2 16 8 31 69

[2단계 : pivot=22]

8 10 16 2 22 30 31 69

[3단계 : pivot=8]

2 8 16 10 22 30 31 69

[4단계 : pivot=16]

2 8 10 16 22 30 31 69

[5단계 : pivot=30]

2 8 10 16 22 30 31 69

- [예제9-3] quickSort() 함수를 수정하여 다음을 프로그램하시오.
 1. 판매실적 자료를 저장하기 위한 구조체 struct sales 를 선언한다. 이 구조체에는 품명과 판매실적 두 개의 멤버변수가 있다. 이 구조체를 새로운 자료형 Sales 로 선언(typedef)한다.
 2. main() 함수에서 Sales 자료형 배열 7개 크기를 선언하고 임의의 값으로 초기화 한다. quickSort_new() 함수를 호출하여 자료를 정렬한다.
 3. quickSort() 함수를 수정하여 quickSort_new() 함수를 작성하시오. 이때 quickSort_new() 의 인수로 int order 를 추가로 넘기고 order=1 이면 판매실적 오름차순으로, order=2 이면 내림차순으로 정렬하여 출력 하시오.

```
Product list[5] = {  
    {"새우깡", 120},  
    {"콜라", 300},  
    {"초코파이", 180},  
    {"사이다", 250},  
    {"감자칩", 90}  
};
```

5. 삽입 정렬

■ 삽입 정렬 insert sort의 이해

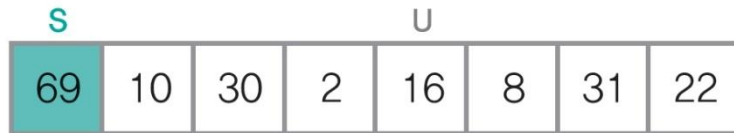
- 정렬되어있는 부분집합에 정렬할 새로운 원소의 위치를 찾아 삽입하는 방법
- 정렬할 자료를 두 개의 부분집합 $S^{\text{Sorted Subset}}$ 와 $U^{\text{Unsorted Subset}}$ 로 가정
 - 부분집합 S : 정렬된 앞부분의 원소들
 - 부분집합 U : 아직 정렬되지 않은 나머지 원소들
 - 정렬되지 않은 부분집합 U 의 원소를 하나씩 꺼내서 이미 정렬되어있는 부분집합 S 의 마지막 원소부터 비교하면서 위치를 찾아 삽입
 - 삽입 정렬을 반복하면서 부분집합 S 의 원소는 하나씩 늘리고 부분집합 U 의 원소는 하나씩 감소하게 함. 부분집합 U 가 공집합이 되면 삽입 정렬이 완성

5. 삽입 정렬

■ 삽입 정렬 수행 과정

- 정렬하지 않은 {69, 10, 30, 2, 16, 8, 31, 22} 자료를 삽입 정렬 방법으로 정렬하는 과정

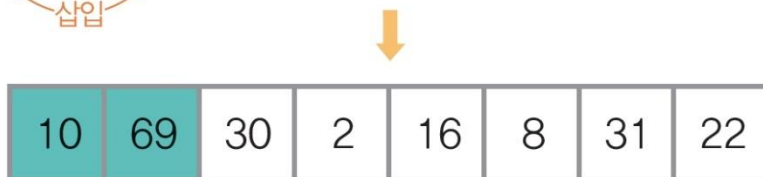
① 초기 상태 : 첫째 원소는 정렬되어 있는 부분집합 S로, 나머지 원소는 정렬되지 않은 원소들의 부분집합 U로 가정



$S = \{69\}$

$U = \{10, 30, 2, 16, 8, 31, 22\}$

① 1단계 : U의 첫째 원소 10을 S의 마지막 원소 69와 비교하면 $10 < 69$ 이므로 원소 10은 원소 69의 앞자리가 됨. 더 이상 비교할 S의 원소가 없으므로 찾은 위치에 원소 10을 삽입



$S = \{10, 69\}$

$U = \{30, 2, 16, 8, 31, 22\}$

5. 삽입 정렬

- ② 2단계 : 현재 U의 첫째 원소 30을 S의 마지막 원소 69와 비교하면 $30 < 69$ 이므로 원소 69의 앞자리 원소 10과 비교. $30 > 10$ 이므로 원소 10과 69 사이에 삽입

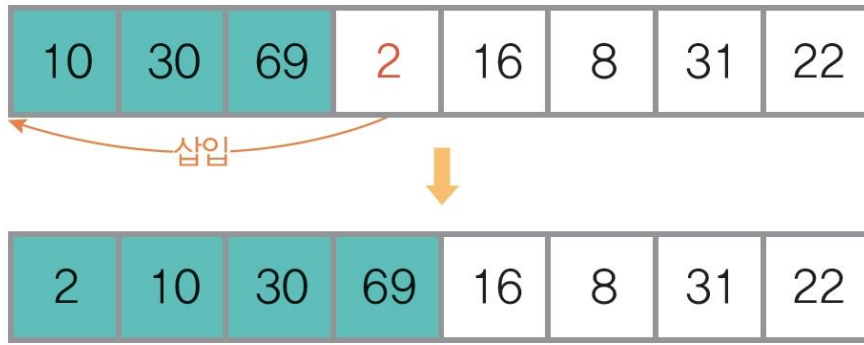


$S = \{10, 30, 69\}$

$U = \{2, 16, 8, 31, 22\}$

5. 삽입 정렬

- ③ 3단계 : 현재 U의 첫째 원소 2를 S의 마지막 원소 69와 비교하면 $2 < 69$ 이므로 원소 69의 앞자리 원소 30과 비교. $2 < 30$ 이므로 다시 원소 30의 앞자리 원소 10과 비교. $2 < 10$ 이고 더 이상 비교할 S의 원소가 없으므로 원소 10 앞에 삽입



$S = \{2, 10, 30, 69\}$

$U = \{16, 8, 31, 22\}$

5. 삽입 정렬

- ④ 4단계 : 현재 U의 첫째 원소 16을 S의 마지막 원소 69와 비교하면 $16 < 69$ 이므로 그 앞자리 원소 30과 비교. $16 < 30$ 이므로 다시 그 앞자리 원소 10과 비교. $16 > 10$ 이므로 원소 10과 30 사이에 삽입

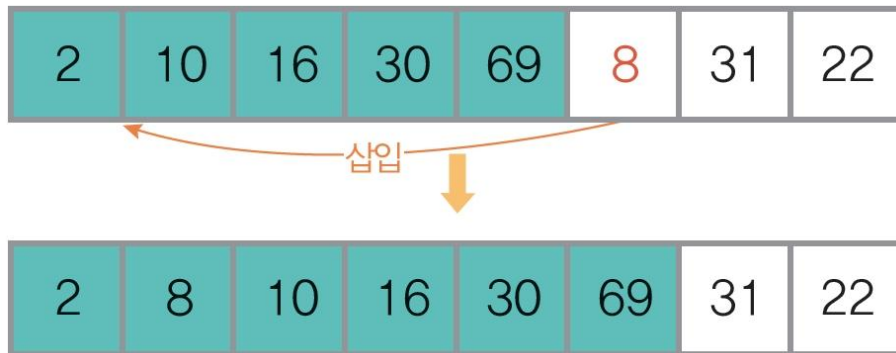


$S = \{2, 10, 16, 30, 69\}$

$U = \{8, 31, 22\}$

5. 삽입 정렬

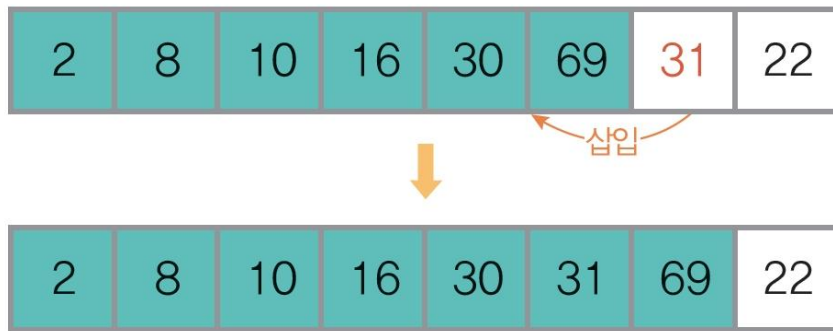
- ⑤ 5단계 : 현재 U의 첫 번째 원소 8을 S의 마지막 원소 69와 비교하면 $8 < 69$ 이므로 그 앞자리 원소 30과 비교. $8 < 30$ 이므로 그 앞자리 원소 16과 비교. $8 < 16$ 이므로 그 앞자리 원소 10과 비교. $8 < 10$ 이므로 다시 그 앞자리 원소 2와 비교. $8 > 2$ 이므로 원소 2와 10 사이에 삽입



$S = \{2, 8, 10, 16, 30, 69\}$
 $U = \{31, 22\}$

5. 삽입 정렬

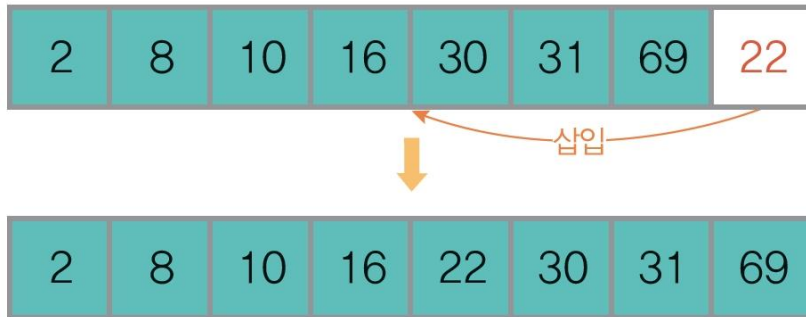
- ⑥ 6단계 : 현재 U의 첫째 원소 31을 S의 마지막 원소 69와 비교하면 $31 < 69$ 이므로 그 앞자리 원소 30과 비교. $31 > 30$ 이므로 원소 30과 69 사이에 삽입



$S = \{2, 8, 10, 16, 30, 31, 69\}$
 $U = \{22\}$

5. 삽입 정렬

- ⑦ 7단계 : 현재 U의 첫째 원소 22를 S의 마지막 원소 69와 비교하면 $22 < 69$ 이므로 그 앞자리 원소 31과 비교. $22 < 31$ 이므로 그 앞자리 원소 30과 비교. $22 < 30$ 이므로 다시 그 앞자리 원소 16과 비교. $22 > 16$ 이므로 원소 16과 30 사이에 삽입. U가 공집합이 되었으므로 실행을 종료



$S = \{2, 8, 10, 16, 22, 30, 31, 69\}$
 $U = \{ \}$

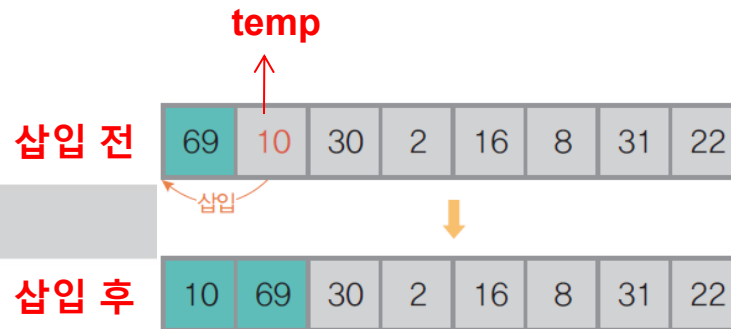
5. 삽입 정렬

- 크기가 n 인 배열 $a[]$ 의 원소를 삽입 정렬하는 알고리즘

알고리즘 9-5 삽입 정렬

```

insertionSort(a[], n)
  for (i ← 1; i < n; i ← i + 1) do {
    temp ← a[i];
    j ← i;
    if (a[j - 1] > temp) then move ← true; // 삽입 자리를 만들기 위해 이동할 원소 표시
    else move ← false;
    while (move and j > 0) do {
      a[j] ← a[j - 1];
      j ← j - 1;
    }
    a[j] ← temp;
  }
end insertionSort()
  
```



$S = \{10, 69\}$

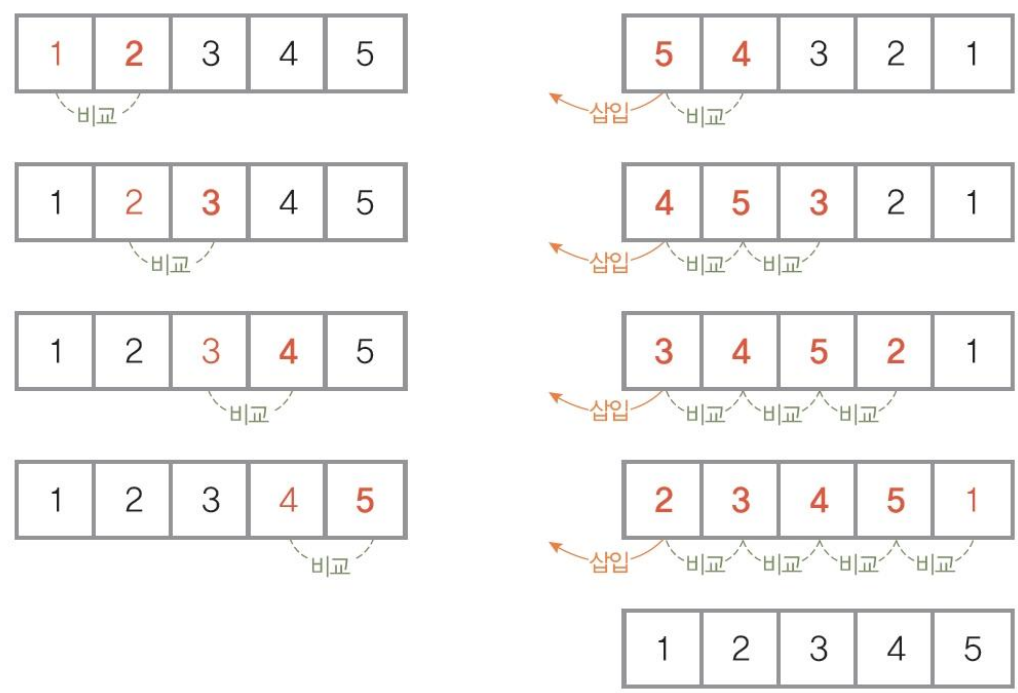
$U = \{30, 2, 16, 8, 31, 22\}$

5. 삽입 정렬

- 메모리 사용공간
 - n개의 원소에 대하여 n개의 메모리 사용
- 연산 시간
 - 최선의 경우 : 원소들이 이미 정렬되어있어서 비교횟수가 최소인 경우
 - 이미 정렬되어있는 경우에는 바로 앞자리 원소와 한번만 비교
 - 전체 비교횟수 = $n-1$
 - 시간 복잡도 : $O(n)$
 - 최악의 경우 : 모든 원소가 역순으로 되어있어서 비교횟수가 최대인 경우
 - 전체 비교횟수 = $1+2+3+ \dots +(n-1) = n(n-1)/2$
 - 시간 복잡도 : $O(n^2)$
 - 최선의 경우와 최악의 경우 평균 비교횟수

$$= \text{avg}(n-1, n(n-1)/2) = (n^2 + n - 2)/4$$
 - 평균 시간 복잡도 : $O(n^2)$

5. 삽입 정렬



- 전체 비교 횟수 : $n-1$
- 전체 자리 이동 횟수 : 0
- 시간 복잡도 : $O(n)$

(a) 순차 정렬된 경우

- 전체 비교 횟수 : $1+2+3+\dots+(n-1) = \sum_{i=1}^{n-1} i$
- 전체 자리 이동 횟수 : $1+2+3+\dots+(n-1) = \sum_{i=1}^{n-1} i$
- 실행 연산 횟수 : $\sum_{i=1}^{n-1} (\text{비교 횟수} + \text{자리 이동 횟수}) = \sum_{i=1}^{n-1} (i + i)$
- 시간 복잡도 : $O(n^2)$

(b) 역순 정렬된 경우

그림 9-4 순차 정렬된 경우와 역순 정렬된 경우의 삽입 정렬 비교

5. 삽입 정렬

- [예제 9-5] 삽입 정렬하기 : [교재 540p](#)
- 실행 결과

정렬할 원소 : 69 10 30 2 16 8 31 22

《《《《《《《《 삽입 정렬 수행 》》》》》》》》》》

1단계: 10 69 30 2 16 8 31 22
 2단계: 10 30 69 2 16 8 31 22
 3단계: 2 10 30 69 16 8 31 22
 4단계: 2 10 16 30 69 8 31 22
 5단계: 2 8 10 16 30 69 31 22
 6단계: 2 8 10 16 30 31 69 22
 7단계: 2 8 10 16 22 30 31 69

예제 9-5 삽입 정렬하기

[insertionSort.c]

```

01  #include <stdio.h>
02
03  void insertionSort(int a[], int size) {
04      int i, j, t, temp;
05
06      for (i = 1; i < size; i++) {
07          temp = a[i];
08          j = i;
09          while ((j > 0) && (a[j - 1] > temp)) {
10              a[j] = a[j - 1];
11              j = j - 1;
12          }
13          a[j] = temp;
14          printf("\n %d단계 : ", i);
15          for (t = 0; t < size; t++) printf("%3d ", a[t]);
16      }
17  }
```

[알고리즘 9-5] 구현

[ex0_5.c]

```

01  #include <stdio.h>
02  void insertionSort(int a[], int size);
03
04  int main(void) {
05      int i, list[8] = { 69, 10, 30, 2, 16, 8, 31, 22 }; // 정렬할 원소
06      int size = sizeof(list) / sizeof(list[0]); // list 배열의 원소 개수
07      printf("\n정렬할 원소 : ");
08      for (i = 0; i < size; i++) printf("%3d ", list[i]); // 정렬 전의 원소 출력
09      printf("\n\n<<<<<<<<<< 삽입 정렬 수행 >>>>>>>>>>\n");
10      insertionSort(list, size); // 삽입 정렬 함수 호출
11
12      getchar(); return 0;
13  }
```

- [예제9-5] insertionSort() 함수를 수정하여 다음을 프로그램하시오.
 - 판매실적 자료를 저장하기 위한 구조체 struct sales 를 선언한다. 이 구조체에는 품명과 판매실적 두 개의 멤버변수가 있다. 이 구조체를 새로운 자료형 Sales 로 선언 typedef)한다.
 - main() 함수에서 Sales 자료형 배열 5개 크기를 선언하고 임의의 값으로 초기화 한다. insertionSort_new() 함수를 호출하여 자료를 정렬한다.
 - insertionSort() 함수를 수정하여 insertionSort_new() 함수를 작성하시오. 이때 insertionSort_new() 는 판매실적 오름차순으로 정렬하여 출력한다.

```
Product list[5] = {  
    {"새우깡", 120},  
    {"콜라", 300},  
    {"초코파이", 180},  
    {"사이다", 250},  
    {"감자칩", 90}  
};
```